

Competencia: 38369 - Controlar la calidad del servicio de software de acuerdo con los estándares técnicos.

RA: 593146 - Incorporar actividades de aseguramiento de la calidad del software de acuerdo con estándares de la industria.

Taller Introductorio QA-00

La importancia de la Calidad del Software

Objetivo: Reflexionar sobre el impacto de la calidad del software mediante el análisis del documental 'Downfall: The Case Against Boeing' (Netflix).

Instrucciones

1. Observe el documental.
2. Tome apuntes de los hechos relacionados con el software, las pruebas, la documentación y la toma de decisiones.
3. Responda las preguntas de forma individual.
4. Socialice las respuestas con su equipo (proyecto formativo) Vídeo.
5. Elabore una reflexión grupal (proyecto formativo).

Preguntas de análisis

1. ¿Cuál fue el problema principal: el avión, el software o ambos? Justifique.

Fue ambos, pero originado por una falla sistémica de gestión y cultura organizacional.

- **El avión (Hardware/Física):** Para competir con Airbus, Boeing colocó motores más grandes y adelantados en el 737 MAX. Esto alteraba la aerodinámica del avión, haciendo que el morro tendiera a elevarse en ciertas maniobras, lo que podía causar una pérdida de sustentación (stall).
- **El software (MCAS):** Se diseñó como un "parche" para corregir la física del avión y hacer que se pilotara igual que la generación anterior. El problema, dependía de un único sensor de Ángulo de Ataque (AoA) un punto único de falla, se activaba repetidamente sin control fácil del piloto y actuaba con demasiada fuerza.

2. ¿Qué decisiones de la empresa aumentaron el riesgo del proyecto?

- **Priorizar velocidad y costo sobre seguridad:** Competir agresivamente contra Airbus llevó a recortar tiempos de desarrollo e inspección.
- **Ocultar el MCAS a los pilotos:** Para evitar que la FAA exigiera costosas capacitaciones en simulador para las aerolíneas, Boeing eliminó casi toda mención del MCAS del manual de vuelo.
- **Diseño sin redundancia:** Confiar una decisión crítica de software a la lectura de un solo sensor en lugar de dos o más.

Competencia: 38369 - Controlar la calidad del servicio de software de acuerdo con los estándares técnicos.

RA: 593146 - Incorporar actividades de aseguramiento de la calidad del software de acuerdo con estándares de la industria.

- **Desactivar alertas visuales opcionales:** La alerta que avisaba si los sensores no coincidían se vendía como un "extra opcional" pagado, en lugar de ser un estándar de seguridad.

3. ¿Qué actividades de aseguramiento de la calidad (QA) considera que no se realizaron o fueron insuficientes?

- **Análisis de Riesgos (FMEA / Failure Mode and Effects Analysis):** No se evaluó adecuadamente el impacto de la falla de un sensor único en combinación con la capacidad de sobreescritura del software.
- **Validación de Requisitos:** No se validó si ocultar el sistema afectaba la usabilidad y la seguridad operativa de los pilotos
- **Auditorías de cumplimiento e independencia en QA:** La FAA delegó la certificación en ingenieros de la misma empresa Boeing (conflicto de interés), debilitando la supervisión objetiva.

4. ¿Qué documentos técnicos debieron existir antes de poner el software en producción?

- **Especificación de Requisitos de Software (SRS):** Documentando detalladamente las reglas de negocio del MCAS (cuántas veces podía activarse, límites de fuerza y fuentes de datos).
- **Matriz de Trazabilidad de Requisitos (RTM):** Para vincular cada riesgo aerodinámico con una prueba de software específica.
- **Manuales de Usuario / Documentación de Operaciones:** Manuales de vuelo que explicaran detalladamente el funcionamiento del MCAS y los procedimientos de emergencia ante una falla.
- **Plan y Reporte de Pruebas de Software (Test Plan & Test Summary Report):** Evidencia firmada de pruebas de estrés, fallas de hardware y redundancia.

5. ¿Qué tipo de pruebas considera que hacían falta?

- **Pruebas de Redundancia y Conmutación por Error (Failover testing):** Probar deliberadamente la desconexión o fallo del sensor AoA para verificar cómo reaccionaba el software.
- **Pruebas Integradas de Simulación (Hardware-in-the-Loop - HIL):** Colocar a pilotos reales en simuladores expuestos al fallo específico del MCAS sin previo aviso, midiendo su tiempo de respuesta y la carga cognitiva.
- **Pruebas de Estrés y Límites:** Probar qué ocurre cuando el software recibe lecturas continuas e erróneas durante un periodo prolongado.

Competencia: 38369 - Controlar la calidad del servicio de software de acuerdo con los estándares técnicos.

RA: 593146 - Incorporar actividades de aseguramiento de la calidad del software de acuerdo con estándares de la industria.

- **Pruebas de Usabilidad / UX en situaciones de crisis:** Evaluar la claridad de las alarmas en cabina.

6. ¿Quién debe responder cuando un fallo crítico llega a producción?

La responsabilidad es compartida, pero recae sobre todo en la alta dirección (C-Level / Gerencia de Proyecto).

La gerencia crea la cultura, impone presiones de tiempo irrealistas y toma las decisiones presupuestarias que recortan la calidad.

Sin embargo, la responsabilidad técnica recae de forma en Cascada (Cascade responsibility) sobre el equipo de arquitectura, desarrollo y QA si no escalaron los riesgos detectados o si aceptaron malas prácticas por presión corporativa.

7. ¿Qué costos produjo la falta de calidad (económicos, legales, humanos y reputacionales)?

- **Costos Económicos:** Decenas de miles de millones de dólares en pérdidas (cancelación de pedidos, indemnizaciones, aviones varados en tierra, multas de entes reguladores).
- **Costos Legales:** Demandas colectivas, investigaciones penales a ejecutivos y acuerdos de sanción con el Departamento de Justicia de EE. UU.
- **Costos Humanos (Irreparables):** 346 vidas humanas perdidas en las tragedias de Lion Air (Vuelo 610) y Ethiopian Airlines (Vuelo 302).
- **Costos Reputacionales:** Pérdida total de confianza por parte de aerolíneas, pilotos, pasajeros y el público general hacia una marca históricamente icono de la aviación.

8. Si usted fuera QA Lead, ¿qué acciones habría exigido antes de aprobar el despliegue?

- **Redundancia obligatoria:** Bloquear el despliegue hasta que el software requiriera la lectura de al menos dos sensores de ángulo de ataque.
- **Criterio de Aceptación con Usuarios Finales:** Exigir que los pilotos de prueba probaran las maniobras de recuperación en simulador con la documentación del sistema completada.
- **Mecanismo de Anulación manual simple:** Garantizar un botón o palanca que desactive de forma inmediata y definitiva la asistencia de software si el piloto lo requiere.
- **Cero Defectos Críticos Abiertos:** No firmar la liberación (Sign-off) mientras existieran riesgos de punto único de falla sin mitigar.

Competencia: 38369 - Controlar la calidad del servicio de software de acuerdo con los estándares técnicos.

RA: 593146 - Incorporar actividades de aseguramiento de la calidad del software de acuerdo con estándares de la industria.

9. Relacione el caso con su proyecto ADSO. ¿Qué riesgos similares podrían presentarse?

En el desarrollo de software web/móvil (proyectos ADSO), aunque no manejamos aviones, pueden presentarse patrones idénticos:

- **Puntos únicos de falla:** Confiar en una sola API externa o servidor sin respaldo de caída.
- **Lógica oculta / Falta de documentación:** Desarrollar funciones críticas (ej. procesos de pago, permisos de seguridad, cálculo de inventario o datos sensibles) sin documentarlas, dejando al usuario final o al administrador a ciegas ante un error.
- **Saltarse pruebas por entregas rápidas:** Presión de tiempo que lleve a desplegar a producción sin hacer pruebas de seguridad, pruebas de carga o validación de entradas de usuario.

10. Escriba tres conclusiones sobre la importancia de planificar la calidad desde el inicio del desarrollo.

- **"Probar al final es corregir, probar desde el inicio es prevenir":** La calidad no es una etapa que se añade al final antes de lanzar a producción; debe ser parte del diseño, la arquitectura y la toma de requisitos (Shift-Left Testing).
- **El costo de corregir un error escala exponencialmente:** Corregir un fallo de lógica en la fase de diseño cuesta una fracción de lo que cuesta arreglarlo en producción, donde los costos pueden ser catastróficos.
- **La documentación y la transparencia son pilares de la calidad:** Un software técnicamente funcional sigue siendo defectuoso si carece de documentación, capacitación para los usuarios y visibilidad sobre sus riesgos operativos.

Discusión grupal

Con su equipo construyan una reflexión (máximo dos páginas) respondiendo: ¿Puede certificarse la calidad de un software que no ha sido planificado para ser evaluado? Sustenten la respuesta con argumentos obtenidos del documental y de su experiencia.

¿Puede certificarse la calidad de un software que no ha sido planificado para ser evaluado?

Nuestra respuesta como equipo es NO. Certificar la calidad de un software significa garantizar con evidencia que este es seguro, confiable y cumple con lo que el usuario

Competencia: 38369 - Controlar la calidad del servicio de software de acuerdo con los estándares técnicos.

RA: 593146 - Incorporar actividades de aseguramiento de la calidad del software de acuerdo con estándares de la industria.

necesita. Si un sistema no se diseña pensando en cómo se va a probar desde el inicio, cualquier "certificación" que se le dé será solo un papel firmado sin valor real, un trámite burocrático que esconde serios riesgos.

Vista desde el documental (Downfall: The Case Against Boeing)

El caso del sistema MCAS en los aviones 737 MAX demuestra lo peligroso que es aprobar un software que no fue diseñado para ser evaluado adecuadamente:

Se ocultó para evitar ser probado: Boeing eliminó la mención del software en los manuales de los pilotos para evitar costos de capacitación. Si algo no se documenta, no se evalúa con los usuarios reales.

Dependencia de un solo punto de falla: El software se diseñó para confiar en la lectura de un solo sensor de ángulo de ataque. Nunca se planificaron pruebas para ver qué pasaba si ese sensor fallaba y enviaba datos incorrectos.

Aprobación por presión y no por rigor: Por la prisa de ganarle a la competencia, las revisiones se hicieron a la ligera. Se "certificó" el sistema en el papel, pero en la práctica no se probaron los escenarios reales de falla, lo que desencadenó las tragedias donde murieron 346 personas.

Conclusión

Certificar un software sin haber planificado su evaluación es una falsa ilusión de seguridad. En el desarrollo de software, la calidad no es algo que se revisa al final, sino algo que se construye desde el primer día. Solo podemos garantizar que un sistema es bueno cuando desde el principio definimos cómo lo vamos a poner a prueba.

Criterios de evaluación

- Argumentación técnica.
- Relación con conceptos de QA.
- Participación en la discusión.
- Calidad de la reflexión escrita.
- Uso de ejemplos del documental.